Extracted PDP Protocols from Reference Schemes

1 Scope and Unified Reading

This note extracts the PDP protocol cores from the four reference PDFs in `refs`. The scheme labels follow the naming style used by `paper/FoldAudit.tex`; the leading "20" is removed from the year in each label.

PDF	Scheme label	PDP protocol core
`ZhangTPDS2023.pdf`	ZhangTPDS23	PCS-MPAP batch auditing over multiple files
`YuTC2025.pdf`	YuTC25	HHF, polynomial commitment, and Tag-IMHT auditing
`XuTIFS2026.pdf`	XuTIFS26	MPA, polynomial commitments plus Merkle authentication
`MiaoSCIS2026.pdf`	MiaoSCIS26	multi-keyword searchable PDP

The notation below preserves each paper's protocol structure. When comparing with FoldAudit, m may denote a file batch, n the number of blocks, s the number of sectors per block, and c the number of challenged blocks. Multi-server and multi-copy components in the source papers are normalized away by taking a single storage server. The extracted formulas therefore keep multi-file auditing behavior while omitting cross-server aggregation and diagnosis logic.

2 ZhangTPDS23: PCS-MPAP Batch Auditing

2.1 Setup and Storage

The system manager samples bilinear-group parameters $(\mathbb{G}_1, \mathbb{G}_2, p, g, e)$, hash functions $H : \{0,1\}^* \to \mathbb{G}_1$ and $H' : \mathbb{G}_1 \to \mathbb{Z}_p$, and publishes

$$\mathsf{Para} = (\mathbb{G}_1, \mathbb{G}_2, g, e, H, H').$$

The user samples $x, \alpha \in \mathbb{Z}_p^*$ and sets

$$sk = (x, \alpha), \qquad pk = (v = g^x, u = g^{\alpha x}, g, g^{\alpha}, \ldots, g^{\alpha^{s-1}}).$$

For each outsourced file F_l, block j is encoded as

$$f_{l,j}(X) = \sum_{k=1}^{s} m_{l,j,k} X^{k-1}.$$

The block authenticator is

$$\sigma_{l,j} = \Big(H(Fid_l \| j) \cdot g^{f_{l,j}(\alpha)} \Big)^x.$$

The storage server checks a received tag by

$$e(\sigma_{l,j}, g) \stackrel{?}{=} e\Big(H(Fid_l \| j) \cdot g^{f_{l,j}(\alpha)}, v \Big).$$

2.2 Efficient Batch Auditing

For a batch of d files, the smart contract samples

$$chal = (\{Q_l\}_{l=1}^{d}, \{r_l\}_{l=1}^{d}, \gamma, z),$$

where $Q_l = \{(a_j, v_j)\}$ is the challenge set for file l, r_l is its evaluation point, and $\gamma, z \in \mathbb{Z}_p$.

For the single storage server, the multi-file aggregated tag is

$$\sigma' = \prod_{l=1}^{d} \left(\prod_{(a_j,v_j)\in Q_l} \sigma_{l,a_j}^{v_j} \right)^{\beta_l}, \qquad \beta_l = \gamma^{l-1} Z_{T\setminus\{r_l\}}(z),$$

where $T = \{r_1, \ldots, r_d\}$ and $Z_S(X) = \prod_{r\in S}(X - r)$. For each file,

$$F_l(X) = \sum_{(a_j,v_j)\in Q_l} v_j f_{l,a_j}(X), \qquad S_l = F_l(r_l).$$

The proof polynomials are

$$\begin{aligned} F(X) &= \sum_{l=1}^{d} \gamma^{l-1} Z_{T\setminus\{r_l\}}(X)\big(F_l(X) - F_l(r_l)\big), \\ H_b(X) &= \frac{F(X)}{Z_T(X)}, \\ L(X) &= \sum_{l=1}^{d} \beta_l \big(F_l(X) - F_l(r_l)\big) - Z_T(z) H_b(X). \end{aligned}$$

The server returns

$$W = g^{H_b(\alpha)}, \qquad W' = g^{L(\alpha)/(\alpha - z)}, \qquad E = \sum_{l=1}^{d} \beta_l S_l,$$

and publishes

$$\mathsf{Proof} = (\sigma', W, W', E).$$

The verifier checks

$$e(\sigma', g) \cdot e(\psi, v) \stackrel{?}{=} e(\zeta, v) \cdot e(W', u \cdot v^{-z}),$$

where

$$\psi = g^{-E} W^{-Z_T(z)}, \qquad \zeta = \prod_{l=1}^{d} \left(\prod_{(a_j,v_j)\in Q_l} H(Fid_l \| a_j)^{v_j} \right)^{\beta_l}.$$

3 YuTC25: HHF, Polynomial Commitment, and Tag-IMHT

3.1 Basic Auditing Protocol

The device selects a secret ψ and publishes polynomial-commitment parameters

$$\Psi_i = g^{\psi^i}, \qquad i \in [0, n].$$

For a chunk $F_i = (F_{i,0}, \ldots, F_{i,n-1})$, define

$$P_{F_i}(x) = F_{i,0} + F_{i,1} x + \cdots + F_{i,n-1} x^{n-1}.$$

The device computes the homomorphic tag

$$\varpi_i = \mathsf{HHF}(P_{F_i}(\psi)),$$

then commits to tags in a Tag-IMHT by hashing $hc_i = C(\varpi_i)$ and publishing the Merkle root h_r.

Given challenge parameters $cp = (k_1, k_2, c)$, the server derives

$$S = \{(id, a_{id})\}, \qquad id = \pi_{k_1}(l), \quad a_{id} = f_{k_2}(l), \quad l \in [1, c],$$

and $z = f_{k_2}(c+1)$. The proof polynomial is

$$P_{\mathsf{prf}}(x) = \sum_{id \in S} a_{id} P_{F_{id}}(x),$$

with quotient

$$Z_{\mathsf{prf}}(x) = \frac{P_{\mathsf{prf}}(x) - P_{\mathsf{prf}}(z)}{x - z}.$$

The server returns

$$\mathsf{prf} = \{Z_{\mathsf{prf}}(x), \{\varpi_i, \Omega_i\}_{i \in S}, P_{\mathsf{prf}}(z)\},$$

where Ω_i is the auxiliary authentication information for the Tag-IMHT path.

The verifier reconstructs h'_r from $\{\varpi_i, \Omega_i\}$, checks $h'_r = h_r$, and computes

$$H_Z = \mathsf{HHF}(Z_{\mathsf{prf}}(\psi)) = \prod_{i=0}^{n-1} \Psi_i^{\rho_i}, \qquad H_{\mathsf{prf}} = H_Z^{\psi} = \prod_{i=1}^{n} \Psi_i^{\rho_{i-1}},$$

where $Z_{\mathsf{prf}}(x) = \sum_{i=0}^{n-1} \rho_i x^i$. Verification checks

$$\prod_{i \in S} \varpi_i^{a_i} \stackrel{?}{=} H_{\mathsf{prf}} \cdot H_Z^{-z} \cdot \mathsf{HHF}(P_{\mathsf{prf}}(z)).$$

3.2 Privacy-Enhanced Verification

The privacy variant changes the proof polynomial. The server samples $\beta \in \mathbb{Z}_p^*$, sets

$$B = \mathsf{HHF}(\beta), \qquad \eta = H(B),$$

and constructs

$$P_{\mathsf{prf}}(x) = \sum_{id \in S} a_{id} P_{F_{id}}(x) + \beta\eta.$$

It then returns

$$\mathsf{prf} = \{Z_{\mathsf{prf}}(x), \{\varpi_i, \Omega_i\}_{i \in S}, P_{\mathsf{prf}}(z), B\}.$$

After the Tag-IMHT check, the verifier accepts iff

$$B^{\eta} \cdot \prod_{i \in S} \varpi_i^{a_i} \stackrel{?}{=} H_{\mathsf{prf}} \cdot H_Z^{-z} \cdot \mathsf{HHF}(P_{\mathsf{prf}}(z)).$$

The mask is a constant term; therefore it affects $P_{\mathsf{prf}}(z)$ but cancels out in the quotient $Z_{\mathsf{prf}}(x)$.

4 XuTIFS26: MPA Auditing

4.1 Setup, Storage, and Tag Verification

A trusted center publishes

$$pp = (\mathbb{G}, \mathbb{G}_T, q, g, \hat{e}, H).$$

The user samples $\alpha \in \mathbb{Z}_q^*$ and publishes

$$pk = (g, g^{\alpha}, g^{\alpha^2}, \ldots, g^{\alpha^{s-1}}).$$

A file F_l is split into n blocks with s sectors:

$$F_l = \{m_{l,j,k}\}_{1 \le j \le n,\, 1 \le k \le s}.$$

Each block $m_{l,j}$ is encoded as

$$f_{l,j}(X) = \sum_{k=1}^{s} m_{l,j,k} X^{k-1},$$

and tagged by the KZG-style commitment

$$\sigma_{l,j} = g^{f_{l,j}(\alpha)}.$$

The tags are hashed as Merkle leaves $leaf_{l,j} = H(\sigma_{l,j})$, and the root $Root_l$ is published on-chain.

To validate received storage, the server recomputes

$$C_{l,j} = \prod_{k=1}^{s} (g^{\alpha^{k-1}})^{m_{l,j,k}} = g^{f_{l,j}(\alpha)}$$

and accepts only if the Merkle root built from $H(C_{l,j})$ matches $Root_l$.

4.2 Challenge, Response, and Verification

The blockchain commit-and-reveal challenge returns

$$Chal = (\{a_j, v_j\}_{j=1}^{c}, r),$$

where $a_j \in [1, n]$, $v_j \in \mathbb{Z}_q^*$, and $r \in \mathbb{Z}_q^*$.

For file F_l, the server computes

$$f_{M_l}(X) = \sum_{j=1}^{c} v_j f_{l,a_j}(X), \qquad val_l = f_{M_l}(r),$$

$$h_l(X) = \frac{f_{M_l}(X) - val_l}{X - r}, \qquad w_l = g^{h_l(\alpha)}.$$

Equivalently, if $h_l(X) = \sum_k h_l[k] X^k$, then

$$w_l = \prod_k (g^{\alpha^k})^{h_l[k]}.$$

The per-file proof is

$$P_l = (val_l, w_l, \{\sigma_{l,a_j}, MerklePath_{l,a_j}\}_{j=1}^{c}).$$

For a batch of T files on a single server, the verifier aggregates the $N = 1$ specialization of the original multi-copy and multi-provider formulation:

$$\sigma_{\mathsf{global}} = \prod_{l=1}^{T} \prod_{j=1}^{c} \sigma_{l,a_j}^{v_j}, \qquad val_{\mathsf{global}} = \sum_{l=1}^{T} val_l, \qquad w_{\mathsf{global}} = \prod_{l=1}^{T} w_l.$$

The multi-file KZG check is

$$\hat{e}(\sigma_{\mathsf{global}}/g^{val_{\mathsf{global}}}, g) \stackrel{?}{=} \hat{e}(w_{\mathsf{global}}, g^{\alpha - r}).$$

The verifier also checks the supplied Merkle path for every challenged block against the corresponding $Root_l$. Multi-copy and multi-provider fallback diagnosis from the original MPA setting is omitted because the comparison model sets $N = 1$.

5 MiaoSCIS26: Multi-Keyword Searchable PDP

5.1 Initialization, Indexes, and Authenticators

The protocol starts from v files and keyword set

$$W = \{w_1, \ldots, w_K\}.$$

The data owner initializes an $a \times a$ matrix $I_{a\times a}$, where a is chosen significantly larger than both v and K ($a \gg v$ and $a \gg K$) to hide the true numbers of files and keywords. Rows encode keyword membership vectors and columns encode files.

The data owner publishes bilinear parameters

$$param = \{\mathbb{G}_1, \mathbb{G}_T, p, g, u, y, pk, e, H_1, H_2, H_3, h_1, f_{key}, \psi_{key}, \pi_{key}, \mathsf{ssg}_{sk}\},$$

where $x \in \mathbb{Z}_q^*$ is the secret key and $y = g^x$. For file F_i with keyword set $W_{F_i} = \{w_{b_1}, \ldots, w_{b_{t_i}}\}$, define

$$H_{F_i} = \prod_{l=1}^{t_i} H_1(w_{b_l}).$$

The match index is

$$\Omega_i = (H_{F_i} \cdot H_2(FID_i))^{-x}.$$

The blinded row address for keyword w_k is $\pi_o(w_k)$, and the encrypted row is

$$ev_{\pi_o(w_k)} = v_{w_k} \oplus f_l(\pi_o(w_k)).$$

For data blocks $m_{i,j}$, the data owner creates index-switch values $\theta_{i,j} = (j, au_{i,j})$ and authenticators

$$\sigma_{i,j} = (H_{F_i} \cdot H_2(FID_i) \cdot H_3(au_{i,j}) \cdot u^{m_{i,j}})^x.$$

The cloud can verify the uploaded authenticators by checking the aggregate pairing relation

$$e\left(\prod_{i=1}^{v}\prod_{j=1}^{n} \sigma_{i,j}, g\right) \stackrel{?}{=} e\left(\prod_{i=1}^{v}\prod_{j=1}^{n} H_{F_i} H_2(FID_i) H_3(au_{i,j}) u^{m_{i,j}}, y\right).$$

5.2 Trapdoor, Challenge, Proof Generation, and Verification

For target keywords $W' = \{w_1, \ldots, w_h\}$, the data owner sends

$$T_w = \{(\pi_o(w_i), f_l(\pi_o(w_i)))\}_{i=1}^{h}$$

to the auditor. The auditor samples $k_1, k_2 \in \mathbb{Z}_q^*$ and challenge count c, and sends

$$Chal = (T_w, c, k_1, k_2)$$

to the cloud.

Both auditor and cloud derive challenged block positions

$$Q = \{j = \psi_{k_1}(\beta)\}_{\beta=1}^{c}$$

and per-file coefficients $v_{i,j} = \pi_{\pi_{k_2}(i)}(j)$. Using the trapdoor, they decrypt matching index rows and derive the union of files that contain at least one queried keyword. This is the source construction's OR-style matching semantics: $R = R_1 \cup \cdots \cup R_\gamma$ on the cloud side and $T = T_1 \cup \cdots \cup T_\zeta$ on the auditor side. Let R denote the union set found by the cloud and T the union set found by the auditor.

The cloud computes the aggregate data value and aggregate authenticator

$$\mu = \sum_{i \in R} \sum_{j \in Q} v_{i,j} m_{i,j}, \qquad T_{auth} = \prod_{i \in R} \prod_{j \in Q} \sigma_{i,j}^{v_{i,j}}.$$

It samples $r \in \mathbb{Z}_q^*$, sets $R_u = u^r$, and masks the aggregate data as

$$\mu_1 = \mu - r h_1(T_w, R_u).$$

The proof is

$$\mathsf{Proof} = (T_{auth}, \mu_1, R_u).$$

Implementation note. The original MiaoSCIS26 text writes $R = g^r$. For a reproducible implementation we use $R_u = u^r$, so that the masking term is algebraically consistent with the data factor $u^{m_{i,j}}$ inside each authenticator:

$$u^{\mu_1} R_u^{h_1(T_w,R_u)} = u^{\mu - r h_1(T_w,R_u)} (u^r)^{h_1(T_w,R_u)} = u^{\mu}.$$

Thus $R_u = u^r$ is an implementation-oriented consistency correction rather than the paper's literal notation.

The auditor retrieves the match indexes Ω_i and authenticator indexes $au_{i,j}$ for $i \in T$ and $j \in Q$. It accepts iff

$$e\left(T_{auth} \cdot \prod_{i \in T} \Omega_i^{\sum_{j \in Q} v_{i,j}}, g\right) \stackrel{?}{=} e\left(\left(\prod_{i \in T} \prod_{j \in Q} H_3(au_{i,j})^{v_{i,j}}\right) \cdot u^{\mu_1} \cdot R_u^{h_1(T_w,R_u)}, y\right).$$

This equation simultaneously checks the matched-file set, challenged authenticators, and the masked aggregate data value.

5.3 File Dynamics

To add a file F_z, the data owner inserts the new file column, namely the $(v+1)$-th column, in $I_{a \times a}$, recomputes the encrypted affected rows, calculates

$$H_{F_z} = \prod_{w \in W_{F_z}} H_1(w), \qquad \Omega_z = (H_{F_z} H_2(FID_z))^{-x},$$

and generates

$$\sigma_{z,j} = (H_{F_z} H_2(FID_z) H_3(au_{z,j}) u^{m_{z,j}})^x.$$

The blockchain receives $\{add, FID_z, S_z, EI', \Omega_z\}$, while the cloud receives $\{\phi_z, \{m_{z,j}\}\}$ and verifies the uploaded authenticators.

To delete file F_z, the data owner zeros the corresponding matrix column, reencrypts the affected rows to form EI', publishes $\{EI'\}$ on-chain, and sends

$$\{delete, FID_z, \mathsf{ssg}_{sk}(delete \| FID_z)\}$$

to the cloud for authenticated deletion.

6 Comparison Pointers for FoldAudit

The four extracted protocols motivate the comparison dimensions used in the FoldAudit paper. ZhangTPDS23 has distinct per-file evaluation points in the PCS-MPAP batch proof but exposes an unmasked linear combination of challenged evaluations. YuTC25 masks the proof polynomial by adding a constant term, but the quotient polynomial still reveals the non-constant part of the challenged aggregate. XuTIFS26 supports multi-file aggregation and Merkle-authenticated updates, while its global check uses a shared evaluation point. MiaoSCIS26 provides keyword-filtered multi-file PDP with data masking, but it is not a polynomial-opening batch protocol.